

Monte Carlo Tree Search for Tic-Tac-Toe and Connect Four

Yizhi Liu, Rui Huang

April 21, 2025

Abstract

Your abstract.

1 Introduction

Monte Carlo Tree Search (MCTS) is a search algorithm that is used for decision making in planning problems. The core idea of MCTS is to build a tree using random simulations to evaluate the unexplored moves. At each iteration, the MCTS goes through four phases:

1. **Selection** – Starting from the root, the algorithm selects the child nodes based on a balance of exploration and exploitation.
2. **Expansion** – Once a node with unexplored moves is found, one of those moves is used to create a new child node.
3. **Simulation** – A randomized game is played from the new node until a terminal state is reached.
4. **Backpropagation** – The result of the simulation is propagated back up the tree to the root to update statistics such as win count and visit count.

This approach allows the algorithm to make the most promising next move given the state of the game. MCTS has been successfully applied to games such as Go, Chess, and general game playing, as well as AI.

Two games would be implemented with MCTS for decision making as an example. 1. TicTacToe is a two-player game played on a 3x3 grid, where players alternate turns placing marks (X or O) with the goal of aligning three in a row. 2. Connect-4 is a two-player turn-based game played on a 6 row by 7 column grid. Players take turns dropping colored discs into one of the columns. The disc falls to the lowest available space within the column. The objective is to be the first to form a horizontal, vertical, or diagonal line of four of one's own discs.

Despite simplicity, both games offer a finite number of possible states, a turn structure, and deterministic outcomes. It is ideal to use them to test AI strategies such as MCTS.

The implementation of MCTS for TicTacToe and Connect-4 aims to demonstrate the effectiveness of the algorithm in exploring optimal strategies. Furthermore, TicTacToe and Connect-4 serve as a base for more complex board games, such as Othello, which can be implemented using the similar MCTS framework.

2 Monte Carlo Tree Search (MCTS)

Monte Carlo Tree Search operates by incrementally constructing a game tree, using random simulations and feedback. The algorithm repeats four steps:

1. Selection

In the selection phase, the algorithm traverses the existing tree from the root node to a leaf node by repeatedly selecting child nodes with the highest potential. This potential is evaluated using the Upper Confidence Bound for Trees (UCT) formula:

$$UCB = w/n + C \cdot \sqrt{\frac{\ln N}{n}} \quad (1)$$

w is the number of wins for child n is the number of simulations (visits) for child N is the number of simulations for the parent node, C is the exploration constant (in this case is $\sqrt{2}$).

2. Expansion

Once a node with unvisited children is reached, the algorithm selects one of the unexplored moves and adds a new node to the tree. This newly created node represents a game state that has not yet been simulated and would be simulated in the next step.

3. Simulation

A simulation is run from the newly added node to a terminal game state (usually a random choice). This phase does not play the game out to the end to estimate the outcome of the move. The result of the simulation (win, loss, or draw) provides data for evaluation of that game path.

4. Backpropagation

The result of simulation is propagated back up the tree. Each node along the path from the simulated node to the root updates its statistics: The number of playouts (visits) is incremented. The number of wins is updated based on whether the outcome was favorable for the player associated with the node.

Backpropagation ensures that nodes representing better outcomes (stronger strategies) are more likely to be selected in future iterations. After backpropagation, the algorithm repeat itself using the updated data from the first step Selection.

3 Application of MCTS to TicTacToe

Applying Monte Carlo Tree Search to the game of TicTacToe involves modeling each possible game state as a node in a tree, where each node represents a specific board state. The MCTS algorithm builds this tree dynamically during its search process, using random simulations to evaluate potential future game states. Here's how each step of MCTS is done for TicTacToe:

1. Selection

In the selection phase, the algorithm begins at the root node, which represents the current game state. It navigates the tree by selecting child nodes according to the Upper Confidence Bound for Trees (UCT) value. In this case, the child node represent mark on one of the empty tile. The algorithm may select the tile with highest win rate or explore tile with low payout.

2. Expansion

Once a node with unvisited children is reached, the algorithm selects one of the unexplored moves and adds a new node to the tree. This newly created node represents a game state that has not yet been simulated and would be simulated in the next step. For example, the starting state for the game would be an empty board. It has nine options to add to the tree, which represent marking on one of the empty grid from a 3x3 grid.

3. Simulation

A simulation is run from the newly added node to a terminal game state (usually a random choice). Tic-Tac-Toe terminal when three same marks horizontally, vertically, or diagonally (win or lose) or no empty grid (draw).

4. Backpropagation

The result of simulation is propagated back up the tree. Each node along the path from the simulated node to the root updates its statistics: The number of playouts (visits) is incremented. The number of wins is updated based on whether the terminal outcome was win/lose/draw for the player associated with the node.

After backpropagation, the algorithm repeat itself using the updated data from the first step Selection.

3.1 Unit Test

To test if the algorithm work (choose the action with highest winrate), a unit test to check the first move of the MCTS is implemented. The estimated first move is mark in the middle of the 3x3 grid. In this way, the randomly placed marks are more likely to connect than the opponent.

3.2 Output

See sample output from a game for Figure 1 in this section.

```

. . .
. X .
. . .

. . .
. X .
. . 0

. . X
. X .
. . 0

. . X
. X .
0 . 0

. . X
. X .
0 X 0

. . X
0 X .
0 X 0

. X X
0 X .
0 X 0

TicTacToe: winner is: 1
```

Figure 1: Output from a game using MC

4 Application of MCTS to Connect Four

In this project, MCTS has been implemented for Connect 4 as well, a classic two-player game where players take turns dropping discs into a vertical grid. The goal is to connect four discs in a row—horizontally, vertically, or diagonally. Unlike TicTacToe, Connect 4 has a much larger state space and deeper strategic depth, making it a more suitable challenge for evaluating the effectiveness and scalability of MCTS. Here's how each step of MCTS is done for Connect 4:

1. Selection

In the selection phase, starting from the root node, recursively select optimal child nodes based on the Upper Confidence Bound (UCT) until a leaf node is reached. In this case, the child node represent drop on one of the unfilled column. The algorithm may select the column with highest win rate or explore tile with low payout.

2. Expansion

f the leaf node is not terminal, generate one or more child nodes by applying legal moves. For example, the starting state for the game would be an empty board. It has seven options to add to the tree, which represent drop on one of the unfilled column in a 6x7 grid.

3. Simulation

Simulate the game from the new child node using random moves until a terminal state is reached. Connect 4 terminal when four same marks horizontally, vertically, or diagonally (win or lose) or no empty grid (draw).

4. Backpropagation

Propagate the simulation result back through the visited nodes to update statistics like win counts and playouts: The number of playouts (visits) is incremented. Each win assigns a win score of 2 for a win, 1 for a draw, and 0 for a loss, enabling gradual learning over multiple iterations.

4.1 Unit Test

Unit tests were implemented to verify the correctness of core Connect-4 game logic. Specifically, the tests cover: Winning conditions: Horizontal and vertical victories are checked by manually constructing board states. Initial state: An empty board is tested to confirm no false-positive winner detection. State transition: `next()` method is implicitly verified via winner tests. These tests ensure that the `Connect4State` logic correctly identifies terminal conditions, validates the win-check mechanism, and forms a reliable foundation for MCTS decision-making.

4.2 Output

See sample output from a game for Figure 2,3,4,5,6 in this section.

```
PS C:\Users\Lenovo\Desktop\  
ts.mcts.connect4.Connect4'  
Step 1:  
. . . . .  
. . . . .  
. . . . .  
. . . . .  
. . . . .  
. . . . X .  
  
Step 2:  
. . . . .  
. . . . .  
. . . . .  
. . . . .  
. . . . 0 .  
. . . . X .  
  
Step 3:  
. . . . .  
. . . . .  
. . . . .  
. . . . X .  
. . . . 0 .  
. . . . X .  
  
Step 4:  
. . . . .  
. . . . .  
. . . . .  
. . . . X .  
. . . . 0 .  
. . . . 0 X .  
  
Step 5:  
. . . . .  
. . . . .  
. . . . .  
. . . . X .  
. . . . X 0 .  
. . . . 0 X .  
  
Step 6:  
. . . . .  
. . . . .  
. . . . 0 .  
. . . . X .  
. . . . X 0 .
```

Figure 2: Output from a game using MC

```
PROBLEMS 697 OUTPUT

Step 7:
. . . . .
. . . . .
. . . . 0 .
. . . . X .
. . . X 0 .
. X . 0 X .

Step 8:
. . . . .
. . . . .
. . . . 0 .
. . . 0 X .
. . . X 0 .
. X . 0 X .

Step 9:
. . . . .
. . . . .
. . . X 0 .
. . . 0 X .
. . . X 0 .
. X . 0 X .

Step 10:
. . . . .
. . . . .
. . . X 0 .
. . . 0 X .
. . . X 0 .
0 X . 0 X .

Step 11:
. . . . .
. . . . .
. . . X 0 .
. . . 0 X .
. . . X 0 .
0 X X . 0 X .

Step 12:
. . . . .
. . . . 0 .
. . . X 0 .
. . . 0 X .
. . . X 0 .
0 X X . 0 X .
```

Figure 3: Output from a game using MC

```

Step 13:
. . . . .
. . . . . 0 .
. . . . X 0 .
. . . . 0 X .
. . X . X 0 .
. . X . X 0 .
0 X X . 0 X .

Step 14:
. . . . .
. . . . . 0 .
. . . . X 0 .
. . . . 0 X .
. 0 X . X 0 .
0 X X . 0 X .

Step 15:
. . . . .
. . . . . 0 .
. . . . X 0 .
. . X . 0 X .
. 0 X . X 0 .
0 X X . 0 X .

Step 16:
. . . . .
. . . . . 0 .
. . 0 . X 0 .
. . X . 0 X .
. 0 X . X 0 .
0 X X . 0 X .

Step 17:
. . . . .
. . . . . 0 .
. . 0 . X 0 .
. . X . 0 X .
X 0 X . X 0 .
0 X X . 0 X .

Step 18:
. . . . .
. . . . . 0 .
. . 0 . X 0 .
. . X . 0 X .
X 0 X . X 0 .
0 X X 0 0 X .

```

Figure 4: Output from a game using MC

```

Step 19:
. . . . .
. . . . 0 .
. . 0 . X 0 .
. X X . 0 X .
X 0 X . X 0 .
0 X X 0 0 X .

Step 20:
. . . . .
. . 0 . . 0 .
. . 0 . X 0 .
. X X . 0 X .
X 0 X . X 0 .
0 X X 0 0 X .

Step 21:
. . . . .
. . 0 . . 0 .
. X 0 . X 0 .
. X X . 0 X .
X 0 X . X 0 .
0 X X 0 0 X .

Step 22:
. . . . .
. 0 0 . . 0 .
. X 0 . X 0 .
. X X . 0 X .
X 0 X . X 0 .
0 X X 0 0 X .

Step 23:
. . . . .
. 0 0 . . 0 .
. X 0 . X 0 .
X X X . 0 X .
X 0 X . X 0 .
0 X X 0 0 X .

Step 24:
. . . . .
. 0 0 . . 0 .
. X 0 . X 0 .
X X X . 0 X .
X 0 X 0 X 0 .
0 X X 0 0 X .

```

Figure 5: Output from a game using MC

```
Step 25:
. . . . .
. 0 0 . . 0 .
. X 0 . X 0 .
X X X X 0 X .
X 0 X 0 X 0 .
0 X X 0 0 X .

. . . . .
. 0 0 . . 0 .
. X 0 . X 0 .
X X X X 0 X .
X 0 X 0 X 0 .
0 X X 0 0 X .

Winner: 1
```

Figure 6: Output from a game using MC

5 Conclusion

This implementation satisfies the core requirement of building a working Monte Carlo Tree Search for Tic-Tac-Toe, Connect-4. It correctly executes the MCTS phases, manages the game loop, and produces valid output. The modular structure allows easy reuse of MCTS for other games. Optional extensions (such as human-AI play or GUI) can be integrated based on this functional foundation.